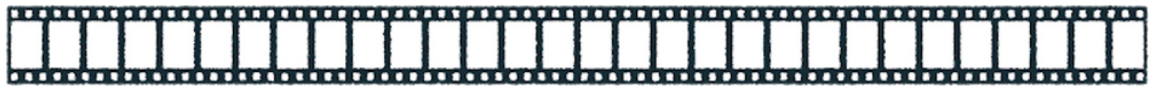
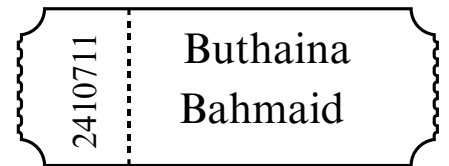
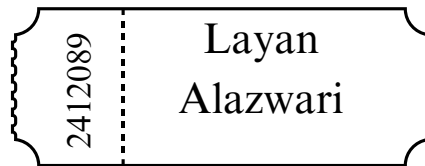
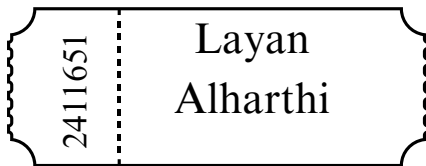


Movies Dataset Analysis



TEAM MEMBERS



COURSE

CEDS-221

Programming for data science

SECTION

DS

PROJECT OUTLINE

This project analyzes the IMDB Top 250 Movies dataset to explore key features like ratings, release years, genres, and directors. The goal is to identify patterns that contribute to a movie's success. We chose this dataset because movies are a popular topic, and analyzing them offers an exciting opportunity to apply data analysis skills.

TABLE OF CONTENT

Dataset Discription & Collection 3

Initial Data Exploration 5

Missing Data Analysis 7

Graphical Exploration 9

Creative Implementation 13

Challenges Faced 16

DATASET DESCRIPTION & COLLECTION



Dataset Name:

IMDB Top 250 Movies Dataset

Source:

The dataset is publicly available on Kaggle.

<https://www.kaggle.com/datasets/rajugc/imdb-top-250-movies-dataset?resource=download>

Number of Records and Columns:

The dataset contains 250 records and 13 columns.

Columns (Description):

- **rank** - Rank of the movie
- **name** - Name of the movie
- **year** - Release year
- **rating** - Rating of the movie
- **genre** - Genre of the movie
- **certificate** - Certificate of the movie
- **run_time** - Total movie run time
- **tagline** - Tagline of the movie
- **budget** - Budget of the movie
- **box_office** - Total box office collection across the world
- **casts** - All casts of the movie
- **directors** - Director of the movie
- **writers** - Writer of the movie

Data Status:

The dataset initially contained missing values and inconsistent formatting, such as numbers combined with special characters like the dollar sign (\$). Therefore, we performed data cleaning steps to handle missing values, remove unnecessary symbols, and standardize the data for accurate analysis.

DATASET DESCRIPTION & COLLECTION



Code:

```
# Task 1: Reading the csv file that contains the dataset
data = pd.read_csv("IMDB Top 250 Movies.csv",
                   na_values=["Not Available",
                              "Unrated", "Not Rated"],
                   thousands=',')

print("**** file is loaded!!****")
```

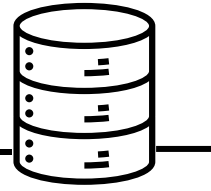
Explanation:

- ✦ In this line, we loaded the movies data from a CSV file using pandas. We told the code to treat some words like "Not Available" or "Unrated" as missing values by using `na_values`.
- ✦ Also, we made sure numbers with commas (like 1,000) are read correctly as numbers by using `thousands`.

Output:

```
In [2]: %runfile C:/Users/Lola/OneDrive/Documents/Project.py --wdir
**** file is loaded!!****
```

INITIAL DATA EXPLORATION



Code:

```
print("\nThe First 5 Rows of the Data:: \n")
print(data.head())

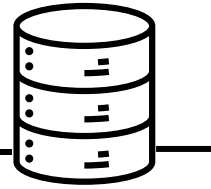
print("\nData Info: \n")
print(data.info())

print("\nData Description: \n")
print(data.describe())
```

Explanation:

- ✦ Relationship between columns:
 - Movies with higher ratings often appear at the top of the rank list (lower rank number = higher rating).
 - The year column shows the movie's release date, which may affect its rating or popularity.
 - Columns like genre, directors, and writers give extra details that may relate to a movie's success or rating.
- ✦ From the summary statistics, we can see that ratings range from 8.0 to 9.3, with an average of 8.3. The year column covers a broad range from 1921 to 2022, showing a mix of old and new movies.
- ✦ Additionally, some columns have missing values, including certificate, run_time, budget, and box_office.

INITIAL DATA EXPLORATION



Output:

1

The First 5 Rows of the Data::

```
rank    ...                               writers
0      1  ...                Stephen King, Frank Darabont
1      2  ...                Mario Puzo, Francis Ford Coppola
2      3  ...  Jonathan Nolan, Christopher Nolan, David S. Goyer
3      4  ...                Francis Ford Coppola, Mario Puzo
4      5  ...                               Reginald Rose
```

[5 rows x 13 columns]

2

Data Description:

	rank	year	rating
count	250.000000	250.000000	250.000000
mean	125.500000	1986.360000	8.307200
std	72.312977	25.125356	0.229081
min	1.000000	1921.000000	8.000000
25%	63.250000	1966.250000	8.100000
50%	125.500000	1994.000000	8.200000
75%	187.750000	2006.000000	8.400000
max	250.000000	2022.000000	9.300000

3

```
<class 'pandas.core.frame.DataFrame'>
```

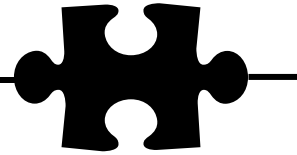
```
RangeIndex: 250 entries, 0 to 249
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	rank	250 non-null	int64
1	name	250 non-null	object
2	year	250 non-null	int64
3	rating	250 non-null	float64
4	genre	250 non-null	object
5	certificate	224 non-null	object
6	run_time	249 non-null	object
7	tagline	250 non-null	object
8	budget	211 non-null	object
9	box_office	220 non-null	object
10	casts	250 non-null	object
11	directors	250 non-null	object
12	writers	250 non-null	object

```
dtypes: float64(1), int64(2), object(10)
```

MISSING DATA ANALYSIS



Code:

```
# Task 3: Checking and Handling missing data
print("Counting all the missing values :\n\n",data.isna().sum())
```

```
data['box_office'] = pd.to_numeric(data['box_office'],errors = 'coerce')
data['box_office']= data['box_office'].fillna(data['box_office'].mean())
```

```
data['budget']= data['budget'].str.replace('$','')
data['budget'] = pd.to_numeric(data['budget'],errors = 'coerce')
data['budget']= data['budget'].fillna(data['budget'].mean())
```

```
data["run_time"] = data["run_time"].fillna("Unknown")

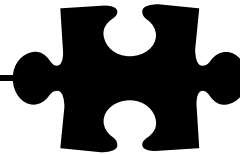
data["certificate"] = data["certificate"].fillna("Unknown")

print("Counting all the missing values after handling them :\n\n"
      ,data.isna().sum())
```

Explanation:

- ✦ We used `.isna().sum()` to find missing values. The columns with missing data were `box_office`, `budget`, `run_time`, and `certificate`.
- ✦ To fix this:
 - We replaced missing values in `box_office` and `budget` with the mean because they are numeric columns and the mean gives a good estimate without losing much information.
 - We filled missing `run_time` and `certificate` with "Unknown" since they are text columns, and this keeps the data complete without deleting rows.

MISSING DATA ANALYSIS



Output:

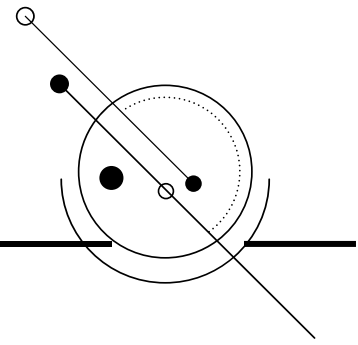
Counting all the missing values :

```
rank          0
name          0
year          0
rating        0
genre         0
certificate   26
run_time      1
tagline       0
budget        39
box_office    30
casts         0
directors     0
writers       0
dtype: int64
```

Counting all the missing values after handling them

```
rank          0
name          0
year          0
rating        0
genre         0
certificate    0
run_time      0
tagline       0
budget        0
box_office    0
casts         0
directors     0
writers       0
dtype: int64
```

GRAPHICAL EXPLORATION



Code:

Bar Visualization

```
# Task 4: Bar - Visualization
genre_boxOffice = grouped_data['box_office'].mean()
genre_boxOffice = genre_boxOffice.sort_values(ascending=False)
genre_boxOffice.plot.bar()
plt.title('Average Box Office per Genre')
plt.xlabel('Genre')
plt.ylabel('Average Box Office')
plt.savefig('BoxOfficeGenre.png', dpi=400)
plt.show()
```

Output:

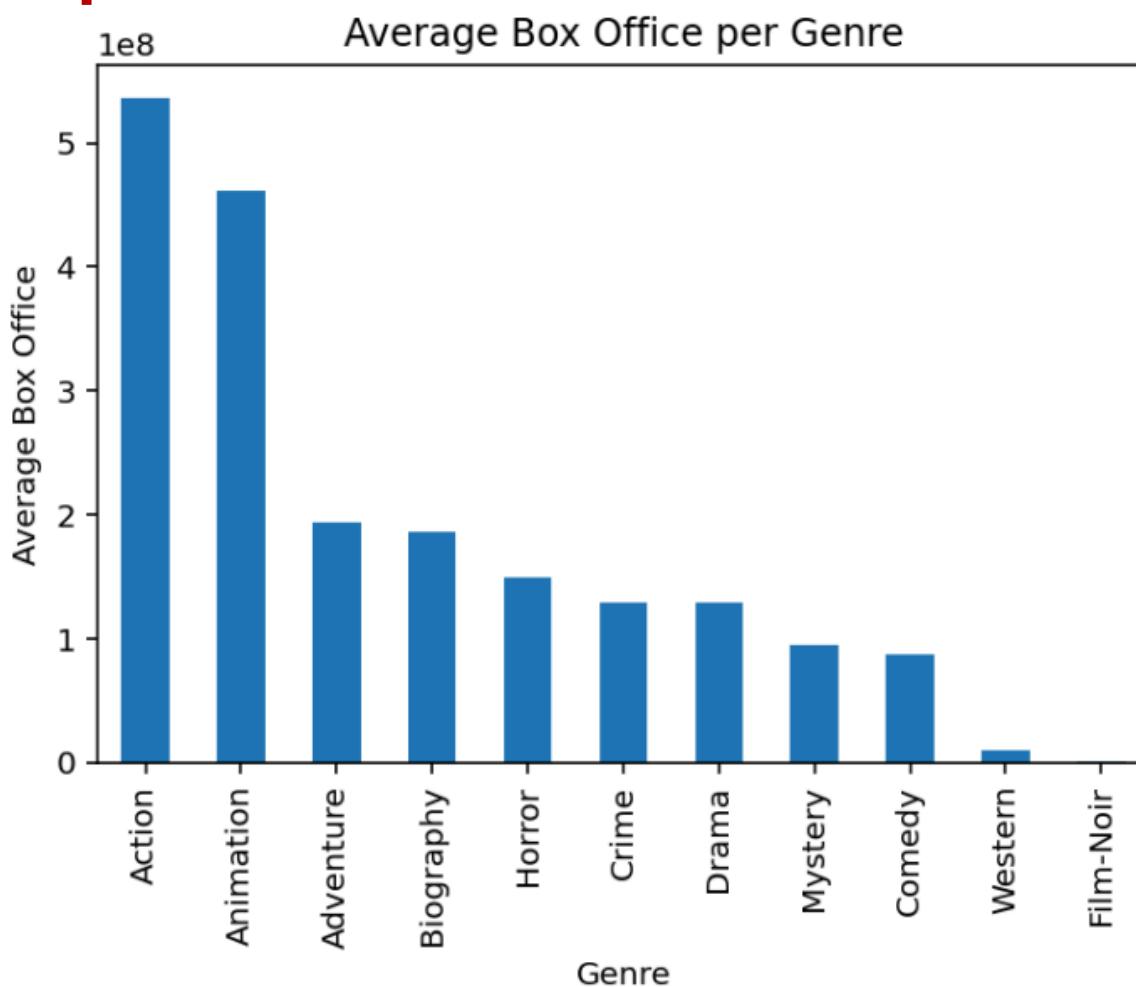
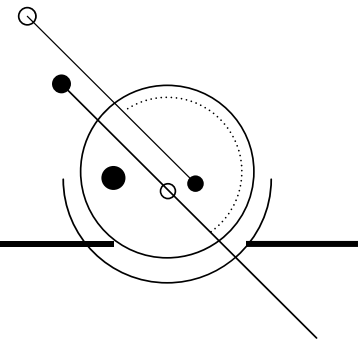


Figure 4.1

GRAPHICAL EXPLORATION



Code:

Histogram Visualization

```
# Task 4: Histogram - Visualization
data["rating"].plot.hist(bins=10, edgecolor='black', figsize=(8,5),
                        title="Distribution of Movie Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Movies")
plt.savefig('MovieRatings.png', dpi=400)
plt.show()
```

Output:

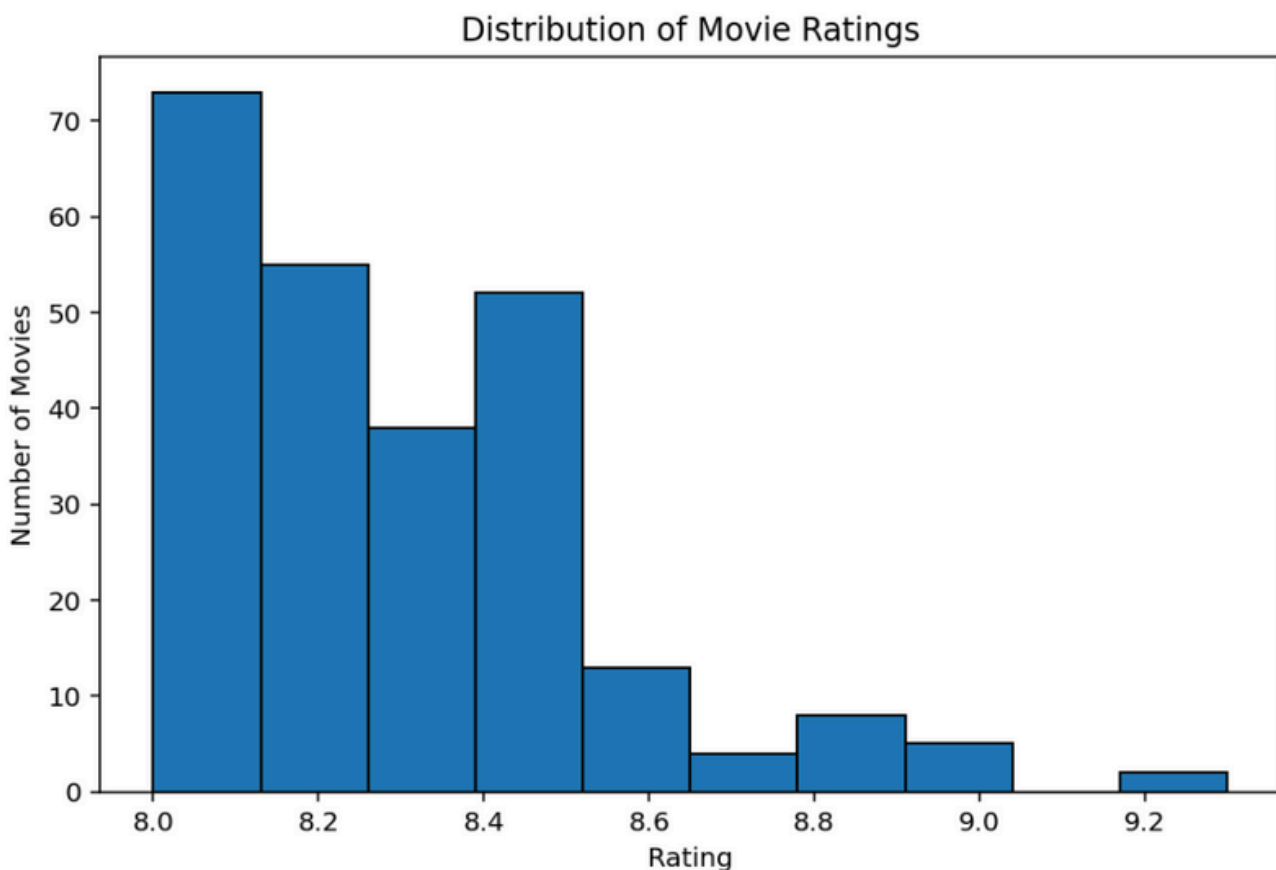
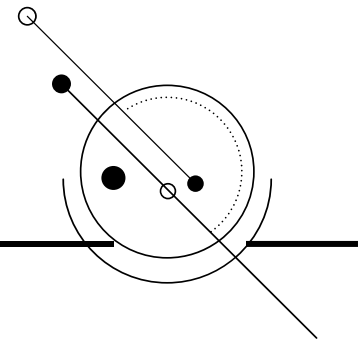


Figure 4.2

GRAPHICAL EXPLORATION



Code:

Line Plot Visualization

```
# Task 4: Line Plot - Visualization
avg_rating = data.groupby('year')['rating'].mean()

plt.figure(figsize=(15, 7))
plt.plot(avg_rating, color='steelblue')
plt.title('Average Movie Ratings Over the Years', fontname='Times New Roman',
         , fontsize=16)
plt.xlabel('Year', fontname='Times New Roman', fontsize=15)
plt.ylabel('Average IMDB Rating', fontname='Times New Roman', fontsize=15)
plt.savefig('AvgRatingOverYears.png', dpi=400)
plt.show()
```

Output:

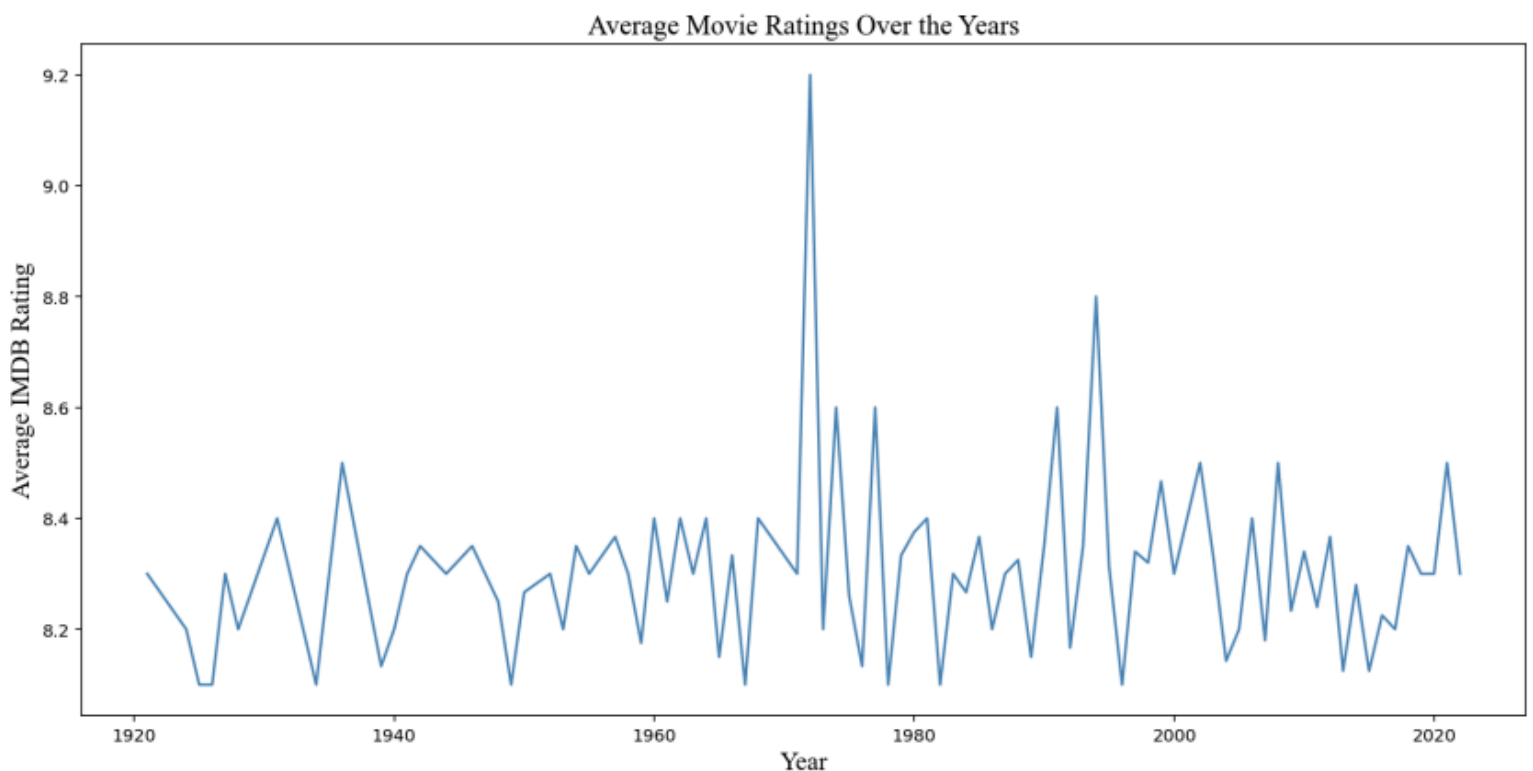
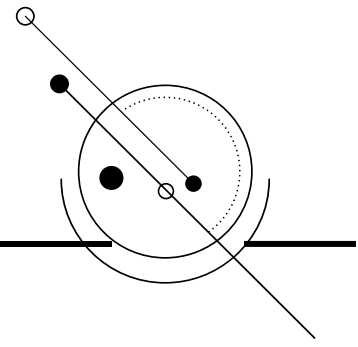


Figure 4.3

GRAPHICAL EXPLORATION



Explanation:

Figure 4.1

- ✦ This bar chart compares the average box office by genre.
- ✦ Action and Animation genres make the most money on average.
- ✦ Genres like Western and Film-Noir have the lowest average, showing not all genres perform equally well at the box office.

Figure 4.2

- ✦ This histogram shows how ratings are distributed among the top 250 movies.
- ✦ Most movies have ratings between 8.0 and 8.5, and only a few movies are rated above 9.0.
- ✦ This means very high ratings are rare, and most top movies have similar good ratings.

Figure 4.3

- ✦ This line chart shows average IMDB movie ratings over the years.
- ✦ There's one big outlier around 1975 where the average rating goes above 9.0 , and another smaller peak around 1995. These high points may be due to very popular movies in those years.
- ✦ Overall, the ratings go up and down, but there's no strong pattern between the year and the rating, suggesting that a movie's rating does not strongly depend on its release year.

CREATIVE IMPLEMENTATION



Code:

Apply & Lambda

```
# Task 5: Creative Implementation
# Number.1
data['movie_age'] = data['year'].apply(lambda x: 'New' if x >= 2010 else 'Old')
print(data["movie_age"])
```

Explanation:

- ✦ The purpose of this code is to add a new column that classify movies as either "New" or "Old" based on their release year. If the movie's year is 2010 or later, it will be labeled as "New," and if it's before 2010, it will be labeled as "Old."
- ✦ Benefits:
 - It quickly and clearly categorizes the data, making analysis easier.
 - The classification ("New" or "Old") can be used to filter or analyze movies based on a specific time period.
 - It saves time and effort by automating the categorization process instead of doing it manually.

Output:

```
Determining whether a movie is considered old or new:
0      Old
1      Old
2      Old
3      Old
4      Old
...
245    New
246    Old
247    Old
248    Old
249    Old
Name: movie_age, Length: 250, dtype: object
```

CREATIVE IMPLEMENTATION



Code:

Apply, Lambda & Groupby

```
# Number.2
data["main_genre"] = data["genre"].apply(lambda x: x.split(",")[0] if pd.notna(x)
                                         else "Unknown")

grouped_data = data.groupby('main_genre')
genre_rating = grouped_data["rating"].mean().sort_values(ascending=False)

print("\n",genre_rating)
```

Explanation:

- ◆ The purpose of this code is to extract the main genre of each movie from the "genre" column (assuming the first genre is the main one), calculate the average rating for each main genre, and then sort the genres by their average rating in descending order.
- ◆ Benefits:
 - It simplifies the process of categorizing movies by their main genre automatically.
 - Provides a quick overview of which genres have the highest average ratings.
 - Helps in better organizing and analyzing the data more efficiently.

Output:

```
main_genre
Crime      8.400000
Action     8.366667
Western    8.350000
Mystery    8.350000
Horror     8.325000
Drama      8.295588
Biography  8.265217
Animation  8.256522
Comedy     8.252174
Adventure  8.222727
Film-Noir  8.100000
Name: rating, dtype: float64
```

CREATIVE IMPLEMENTATION



Code:

Custom Plotting Styles

```
plt.figure(figsize=(15, 7))
```

1

```
plt.plot(avg_rating, color='steelblue')
```

2

```
plt.xlabel('Year', fontname='Times New Roman', fontsize=15)
```

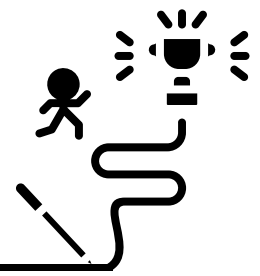
3

Explanation:

- ✦ The purpose of customizing the plot is to make it clearer and more visually appealing. Adjusting things like size, font, and color helps improve the overall look and makes the data easier to understand.
- ✦ Benefits:
 - **Better Readability:** Customizing font size and type makes the labels and title easier to read.
 - **Nicer Look:** Using a color like 'steelblue' and adjusting the plot size makes it look more professional.
 - **Easier Analysis:** A clean and clear plot helps focusing on important trends and patterns in the data.

Output:

Was previously shown in Figure 4.3



1. Implicit Missing Values:

The original dataset didn't have explicit missing values, but it contained placeholders like "Not Available" and "Unrated." We couldn't handle these directly, so we used `na_values` when reading the CSV to convert these values into NaN.

2. Errors in Data:

While processing the data, we encountered errors when converting text to numbers. We used `errors = 'coerce'` in some columns to avoid errors and ensure proper conversion.

3. Handling Non-Numeric Data:

Some columns like "budget" and "box_office " contained values stored as strings with symbols like the dollar sign (\$) . We had to remove symbol before converting the values into numbers, ensuring that the data could be processed correctly.